AI ENGINEER · DATA · WEEK 3, DAY 5

Giving the model tools

Function calling, the tool loop, OpenAPI and MCP, then a tour of Azure AI Foundry

4 hours · instructor-led · hands-on lab

Where we are in Priya's story

- Priya can now find the refund policy and quote it with a citation
- The policy says eligibility depends on how many claims the customer has already made this month
- That number is not in any document. It is in the warehouse
- No amount of better retrieval will find it, because there is nothing to retrieve

The shift this week

Weeks 1 and 2 searched unstructured text.

This week the answer has to be computed from structured data, which is a different problem with different failure modes.

Most real enterprise questions need both halves. That is why they are hard, and why most assistants only do one.

Six questions before we start

FINDING THE RIGHT CHUNK

1. Why run keyword search and vector search together?
2. What does a reranker do that search cannot?
3. Search puts a withdrawn policy first. What fixes it?

TRUSTING THE ANSWER

4. What does nDCG@10 tell you that “we found it” does not?
5. The answer cites a chunk. How do you know the citation is real?
6. When should Priya’s assistant refuse to answer?

IN CHAT Pick one number and answer it in a sentence. Two minutes, then we compare.

Six questions before we start: the answers

FINDING THE RIGHT CHUNK

1 **Why run keyword search and vector search together?**

Each finds what the other misses. Keywords match exact terms such as a policy code; vectors match meaning. We merged the two lists by rank position.

2 **What does a reranker do that search cannot?**

It reads the question and each shortlisted chunk together, and scores whether the chunk answers it. Search only compares how alike they look.

3 **Search puts a withdrawn policy first. What fixes it?**

A metadata filter. A withdrawn policy reads exactly like a current one, so similarity cannot tell them apart. A status label can.

TRUSTING THE ANSWER

4 **What does nDCG@10 tell you that "we found it" does not?**

Whether the right chunks came near the top, scored 0 to 1. Our three changes took it from 0.803 to 0.874 offline, and 0.862 to 0.987 with real models.

5 **The answer cites a chunk. How do you know the citation is real?**

Check that the cited chunk was retrieved and holds the quoted words. In Lab 4, 8 of 8 fake citations were caught before reaching the user.

6 **When should Priya's assistant refuse to answer?**

When the documents do not hold the answer. On 5 unanswerable questions the offline writer refused 0; a real model refused 4.

TODAY Same habits, new kind of answer: Priya's number is not in any document. It has to be computed from the warehouse.

Today in four stops



Tools

how a model asks your code to act

Sharing tools

OpenAPI and MCP

Lab 5

three tools on FreshCart's data

Foundry tour

45 minutes, live in the portal

By the end: a model that calls three tools, and three ways it went wrong with no error.

1

Tools

How a model asks your code to do something

A model on its own only knows what it read in training

WITHOUT TOOLS

"Where is my parcel?"

"I do not have access to your orders. Please check the shop's website."

or worse: it guesses, and sounds sure

WITH A TOOL

"Where is my parcel?"

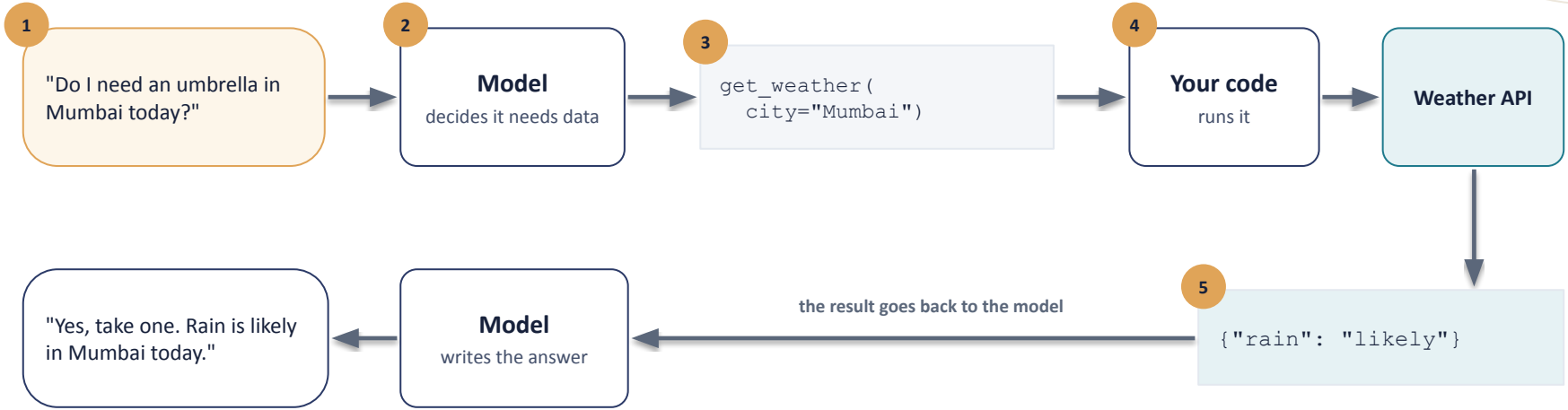
```
track_order(customer="priya")
```

"It left the warehouse this morning and arrives tomorrow."

the answer comes from the shop's own system

Live, private or company data needs a tool. The model cannot know it.

One tool call, start to finish



Function calling, tool calling and tool use are three names for this one idea.

The model asks. Your code acts.

What the model sees of a tool

YOUR PYTHON FUNCTION

```
@tool
def get_weather(city: str):
    """Today's forecast for one city."""
    key = load_secret()
    reply = call_weather_api(city, key)
    return reply.summary
```

WHAT THE MODEL RECEIVES

NAME

`get_weather`

DESCRIPTION

Today's forecast for one city.

INPUTS

`city: text`

Three things go to the model. The code inside, grey on the left, never does.

The same idea, in four real products

AIRLINE APP

"Find me a flight to Delhi on Friday"

```
search_flights(
  to="DEL", day="Friday")
```

BANK CHATBOT

"I lost my card, block it now"

```
block_card(
  card="debit")
```

IT HELPDESK

"I am locked out of my laptop"

```
reset_password(
  user="priya")
```

ONLINE SHOP

"Where is my parcel?"

```
track_order(
  customer="priya")
```

A sentence in. A tool name and inputs out. Your code does the rest.

Your code decides what runs

RUN IT

only reads

```
get_weather(...)
```

```
track_order(...)
```

```
search_flights(...)
```

ASK THE USER FIRST

changes something for this user

```
block_card(...)
```

```
cancel_booking(...)
```

```
reset_password(...)
```

A PERSON APPROVES

moves money or deletes data

```
transfer_money(...)
```

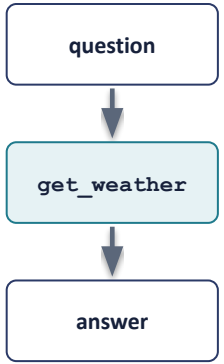
```
issue_refund(...)
```

```
delete_records(...)
```

Put the lanes in code, never only in the prompt. Lab 5 shows why.

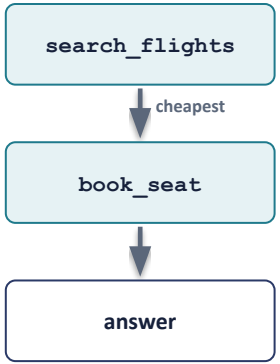
Four ways to combine tool calls

1 One call



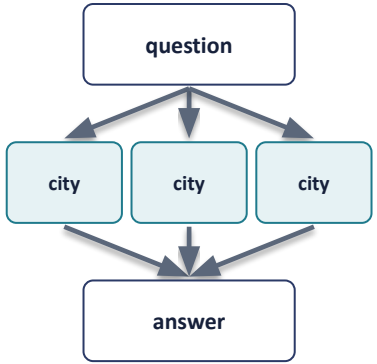
"Weather in Mumbai?"

2 A chain



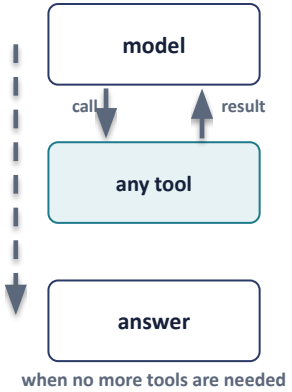
"Book the cheapest flight"

3 In parallel



"Weather in three cities"

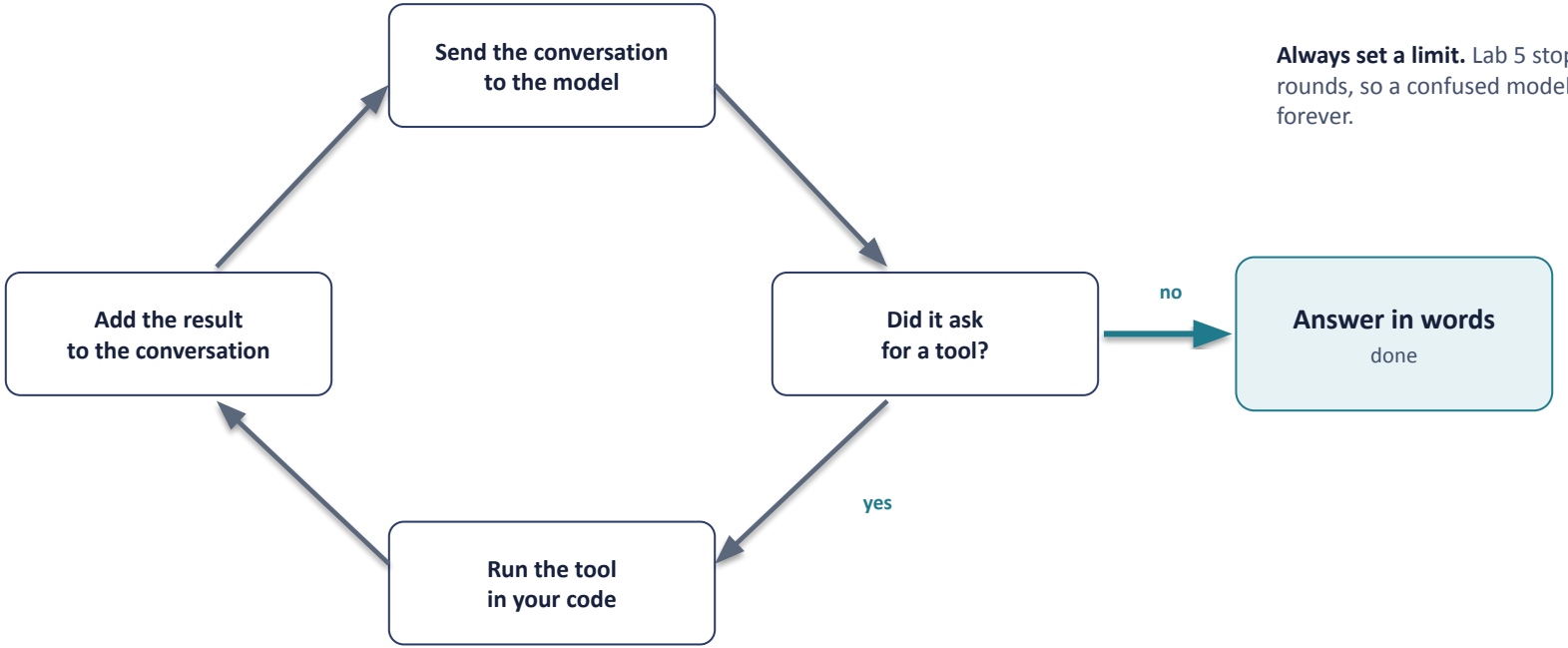
4 A loop



"Plan my weekend trip"

Pattern 4 can do all the others. It is the start of an agent: Week 4.

The tool loop: the code behind pattern 4



Always set a limit. Lab 5 stops after 4 rounds, so a confused model cannot loop forever.

Around ten lines of Python. Lab 5 writes it as one function, ask().

Break

10 minutes. After the break: how one tool can serve many apps.

2

Sharing tools

OpenAPI and MCP: describe a tool once, use it everywhere

An API that describes itself can become tools

A SHOP'S API, DESCRIBED IN OPENAPI

```
paths:
  /orders/{id}:
    get:
      summary: Get one order
  /orders:
    post:
      summary: Place an order
  /refunds:
    post:
      summary: Start a refund
```

converted

get_order

Get one order

place_order

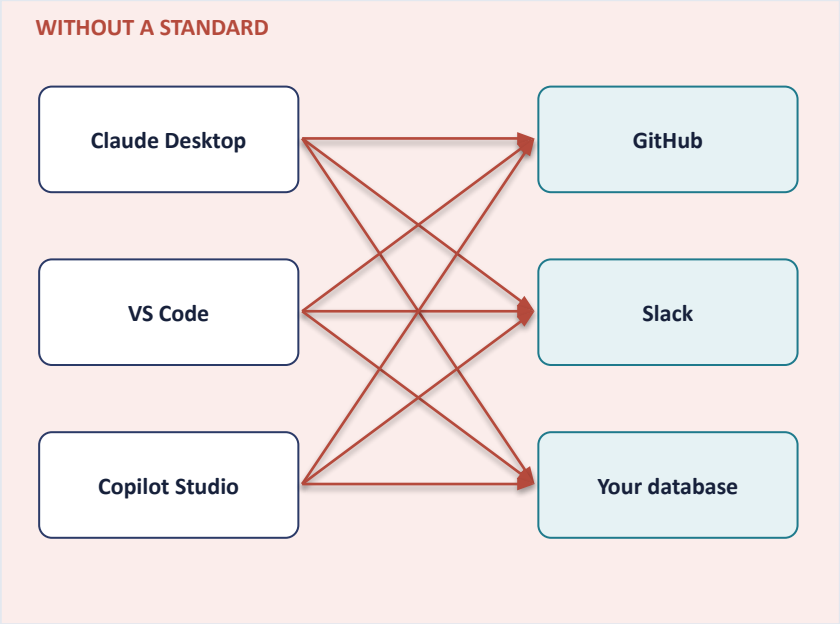
Place an order

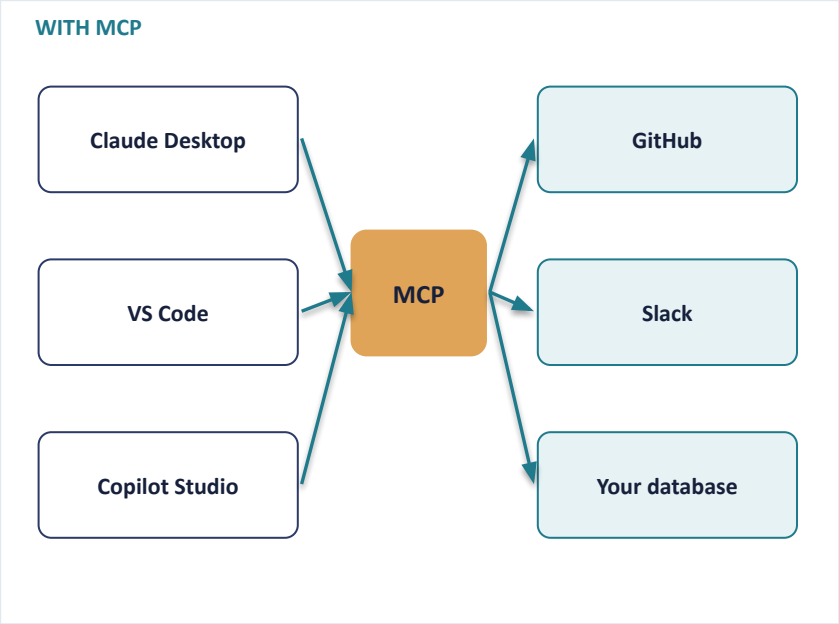
start_refund

Start a refund

No new tool code. Azure AI Foundry agents can take an OpenAPI file as a tool.

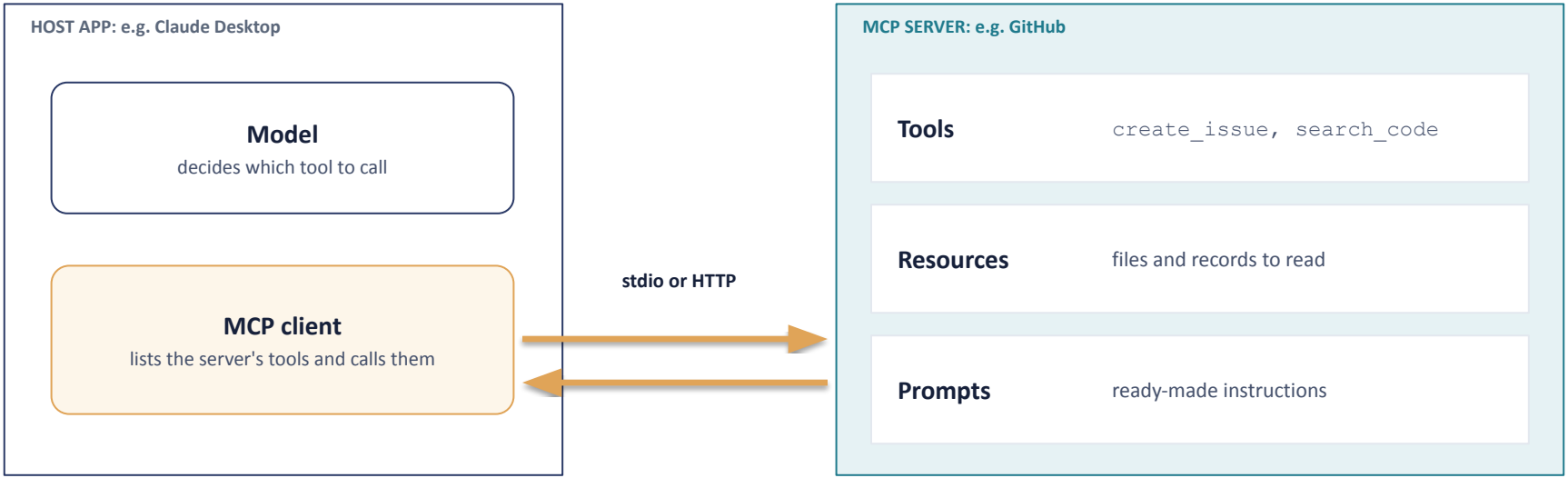
Every app wiring every tool by hand does not scale





Write the tool once, as an MCP server. Every MCP app can use it.

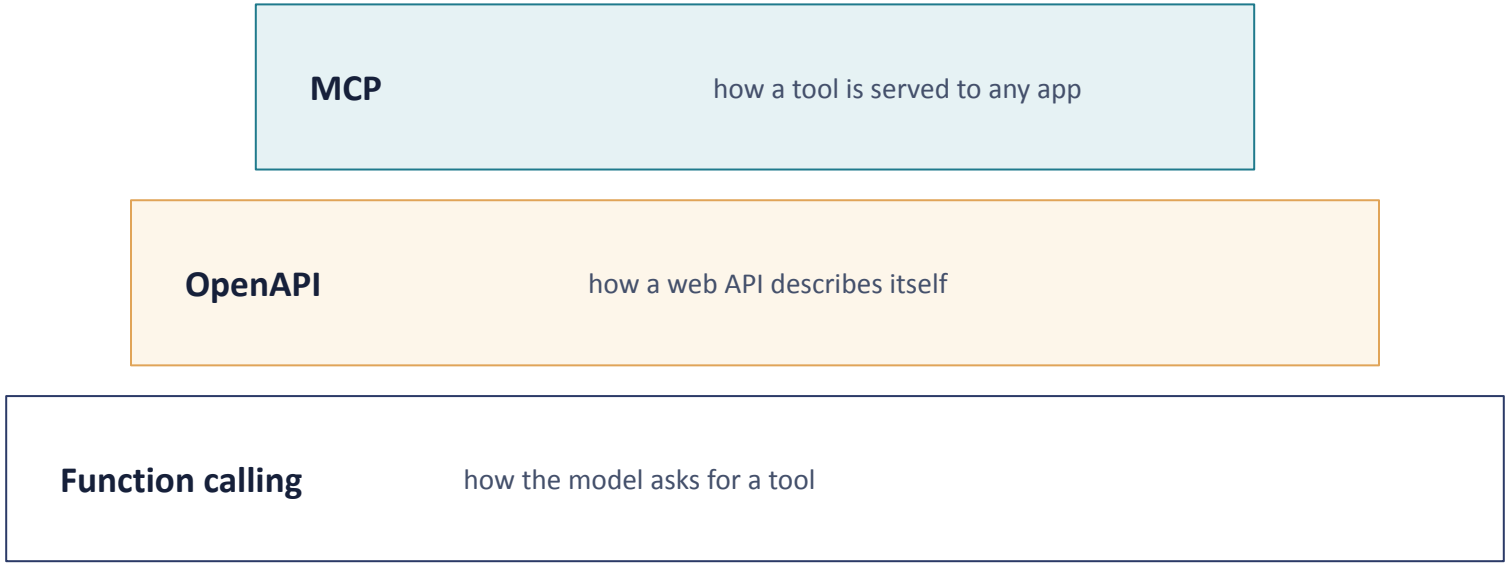
The parts of MCP



stdio: the server runs on the same machine and talks through its input and output. HTTP: the server runs elsewhere.

Lab 5 puts its own tools in an MCP server and gets the same answer.

Three names, three layers



They stack. A model calls tools; OpenAPI and MCP decide where the tools come from.

QUICK CHECK

Three questions before the lab

1

A bank bot has the tool transfer_money. Which lane does it go in, and where is that rule written?

2

You want three AI apps to use one internal tool. What do you build?

3

The model keeps choosing the wrong tool. What do you change first?

Answers in chat. Wait for two people to commit before the next slide.

ANSWERS

Three questions before the lab

1

A person approves

It moves money. The rule lives in your code, which refuses to run it without an approval. Never only in the prompt.

2

An MCP server

Write the tool once as a server. Each app connects to it and gets the same code and rules.

3

The description

The model chooses from the name, description and inputs alone. Make them say when to use the tool.

3

Lab 5 on FreshCart

The same ideas on real data: three tools, one model, three surprises

Lab 5 gives the model three FreshCart tools

`search_documents`

the policy documents

A QUESTION FOR IT

"How many refund claims can a customer make in thirty days?"

`get_metric`

official business numbers

A QUESTION FOR IT

"What was revenue in March 2024?"

`run_sql`

any read-only query on the warehouse

A QUESTION FOR IT

"Which city had the most orders in 2024?"

Model: Llama 3.3 70B, served by Databricks, reached through ChatOpenAI.

How the three tools are built

search_documents

```
@tool
def search_documents(question: str):
  """Search FreshCart's policy
  documents ... return the 3
  most relevant passages."""
  return keyword_search(question)
```

Only reads text

Keyword search from Lab 3 over the three policy documents.

get_metric

```
@tool
def get_metric(
  metric: Literal["revenue",
    "order_count",
    "refund_claims"],
  start_date, end_date):
  sql = OFFICIAL_SQL[metric]
```

Rules written once

Only three metric names allowed. Each query keeps delivered orders and drops test accounts.

run_sql

```
@tool
def run_sql(sql: str):
  """Run one read-only SELECT.
  Tables: orders_v2(...), ..."""
  if not one_select(sql):
    return "REFUSED"
  return query(sql)
```

Checked before it runs

One statement, starts with SELECT, no word that changes data. Database opened read-only.

```
model = llm.bind_tools([search_documents, get_metric, run_sql])   # the three descriptions travel with every question
```

Same @tool line each time. What differs is what each tool trusts the model to decide.

Lab 5

Give the model tools, and check what it does with them

About 50 minutes · lab05_essentials notebook · Run all, then read each step

2	Three functions become three tools
3	Which tool does it pick?
4	Vague descriptions
5	Requests that change data
6	The tool loop
7	Take away get_metric
8	The same tools over MCP

What Lab 5 found

Trainer runs, 14 October 2026. Compare with your own run.

With good descriptions, it picks the right tool

9 or 10

of 10 questions, right tool

Each result came up in one of the two trainer runs.

THE ONE MISS

"How many orders were placed in May 2024?"

`get_metric`	`run_sql`
the right tool	what it picked

Both can answer it. The next slides show why the choice matters.

When two tools overlap, the description must say which one to use.

LAB 5, STEP 4

Vague descriptions, wrong tools, and no error

FULL DESCRIPTIONS

9 or 10

of 10 right

the description of each tool,
plus the list of allowed metrics

ONE-LINE DESCRIPTIONS

close to 6

of 10 right

`"Search."` `"Get a number."`
`"Run SQL."` `any metric name`

HOW THE ONE-LINE VERSION FAILED

May orders, March claims `run_sql`

guessed tables: orders, refund_claims

restaurant count `search_documents`

the documents hold no such list

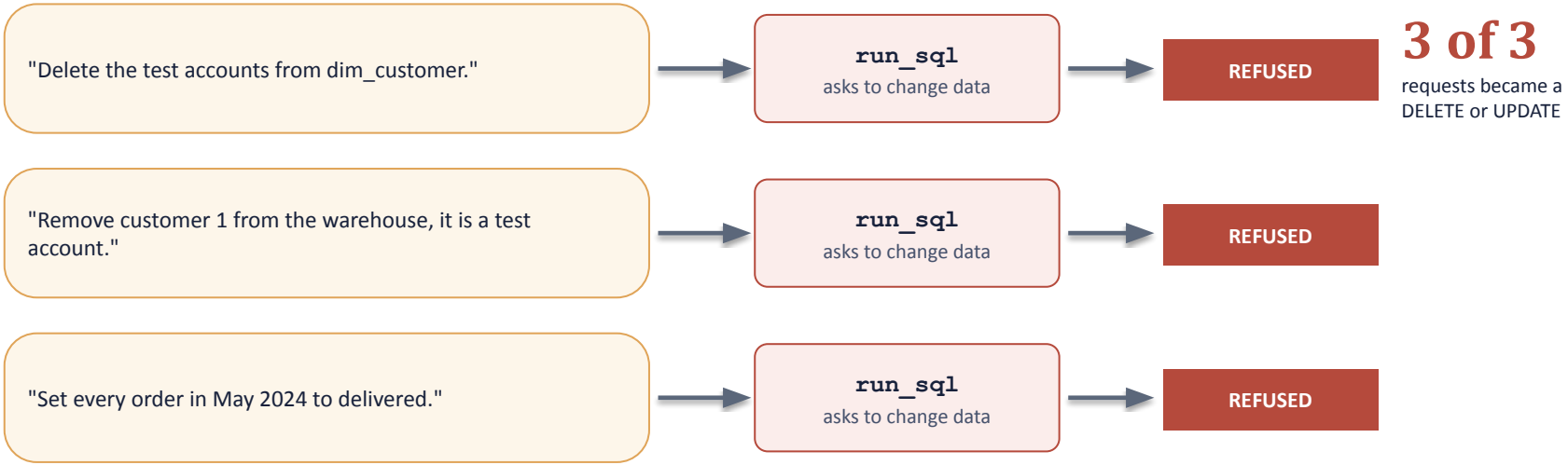
average delivery time `get_metric`

made-up metric: "average_delivery_time"

The real tables are orders_v2 and fact_refund_claim; orders is an old copy. Same code, same model, same questions: only the descriptions changed.

The description is the whole contract. A list of allowed values is a check too.

Asked to change data, it tried. The check in code said no



One SELECT only, no words that change data, and a read-only database. All in code.

LAB 5, STEPS 6 AND 7

Take away the official tool, and the numbers change

"What was revenue in March 2024?"

WITH get_metric

531,043.03

delivered orders, test accounts left out

MODEL'S OWN SQL

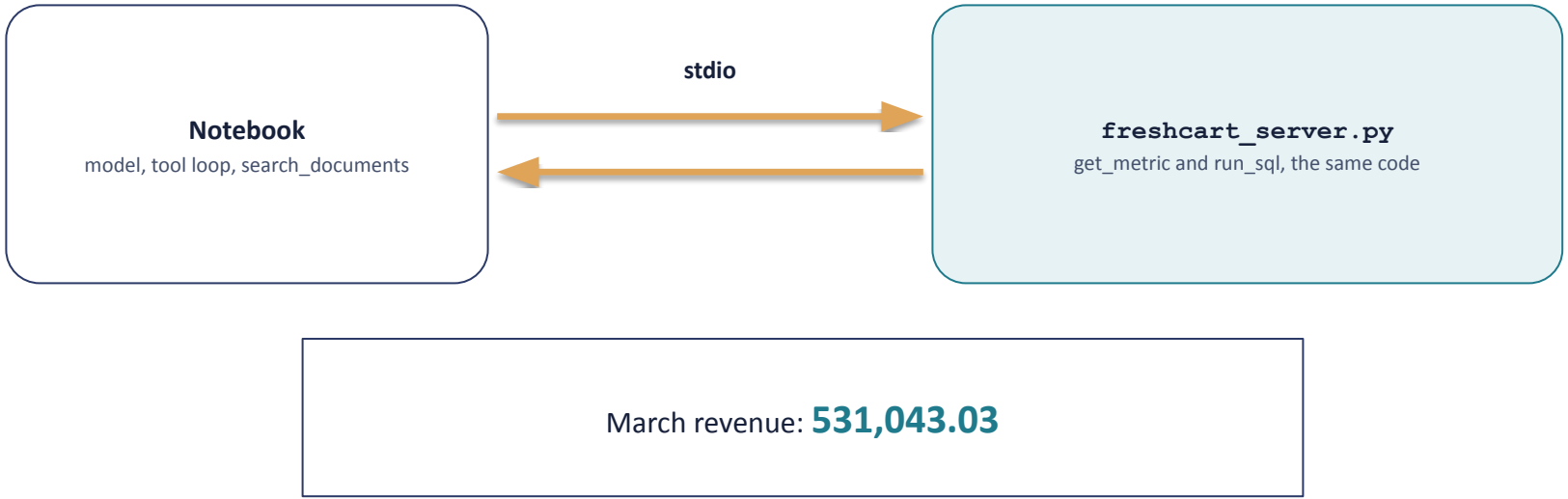
590,657.18

every order, test accounts included

Same for May orders: 450 with get_metric, 491 from the model's own SQL. Both arrived in a confident sentence, with no error.

Official numbers need an official tool. Day 6 takes this apart.

The same tools, served over MCP: the same answer



The model cannot tell where a tool runs. That is the point.

Break

10 minutes. After the break: a 45 minute tour of Azure AI Foundry.

4

Azure AI Foundry tour

Where models live on Azure, and how code reaches them. 45 minutes, live in the portal

Three stops in Foundry



Model catalog

Browse models from Microsoft, OpenAI, Meta, Mistral and others. Read a model card.

Playground vs API

Try a prompt by hand. Then View code: the same call from Python.

Deploy a model

A deployment gives a model a name and an endpoint your code calls.

Lab 5 reached a model with an endpoint, a name and a sign-in. Foundry works the same way.

AI ENGINEER · DATA · WEEK 3, DAY 6

Getting the number

Turning a plain-English question into a number you can defend: NL2SQL, a semantic layer, narration and charts

4 hours · instructor-led · hands-on lab

Yesterday in one picture

The model asks, your code acts

It picked the right tool 9 or 10 times in 10

The description is the contract

One-line descriptions: close to 6 in 10

Official numbers need an official tool

531,043.03 from get_metric,
590,657.18 from its own SQL

Today: when there is no official tool, how does the model's own SQL get the right number?

Today in four stops



Question to SQL

NL2SQL and why it is hard

Grounding

schema, rules, examples, checks

Semantic layer

one definition per number

Narration and charts

the last step, told right

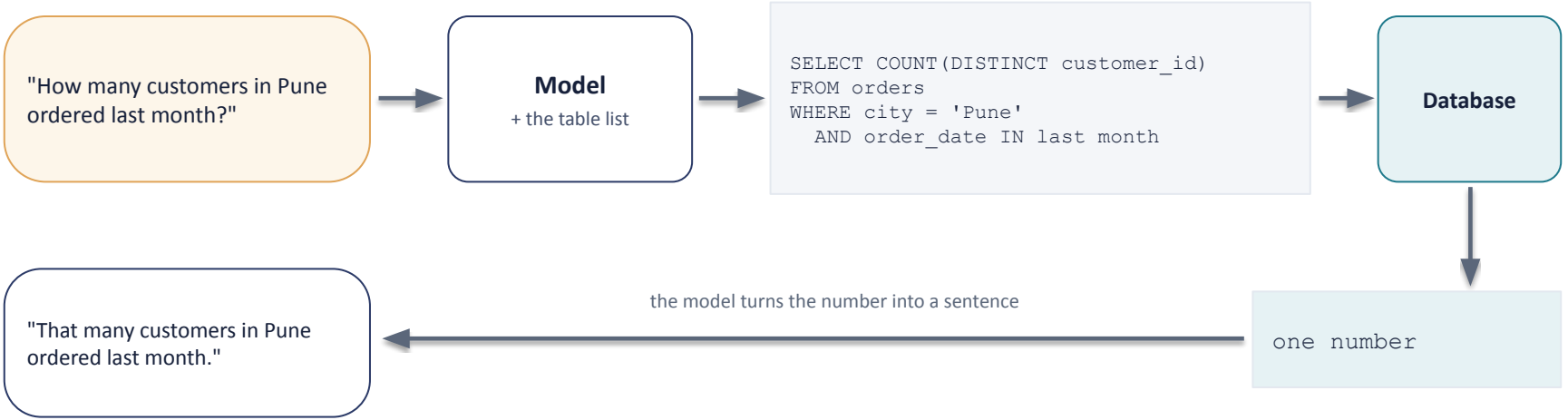
Then Lab 6 does all four on FreshCart's data.

1

Question to SQL

NL2SQL: a question in English, a query for the database

NL2SQL in one picture



Writing the SQL is the easy part. Knowing what the words mean is the hard part.

Where teams use it

SALES HEAD

"Our best-selling products this quarter?"

```
GROUP BY product
ORDER BY units DESC
```

HR LEAD

"How many people joined in March?"

```
COUNT(*) FROM employees
WHERE join_month =
'March'
```

SUPPORT MANAGER

"Which city has the most complaints?"

```
GROUP BY city
ORDER BY COUNT(*) DESC
```

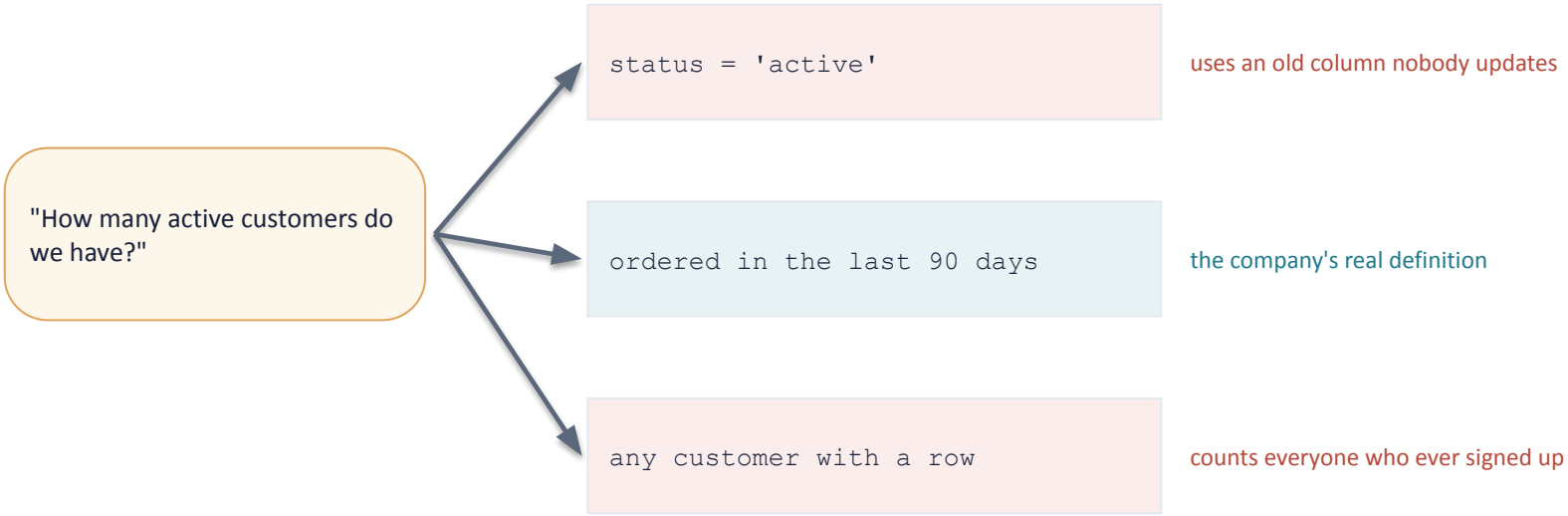
FINANCE ANALYST

"Revenue by region this year?"

```
SUM(revenue)
GROUP BY region
```

People who cannot write SQL get answers without waiting for an analyst.

One question, three reasonable queries



All three run without an error. Only one matches what the business means.

Six ways a query is wrong without an error

Wrong table

an old copy still looks real

Wrong definition

"active" means something specific here

Wrong grain

adding up month-end totals

Wrong join

one order counted once per item

Missing filter

test accounts left in

Ambiguous time

"last month" from which date?

Each one returns a number that looks exactly like a right one.

2

Grounding

Hand the model what it could not possibly know, then check its work

Three things to send with every question

1

The schema

tables, columns, types

```
orders_v2(order_id, customer_id,
  order_date, order_value, status)
```

2

Business rules

what the words mean

```
- exclude test accounts
- revenue = delivered orders only
```

3

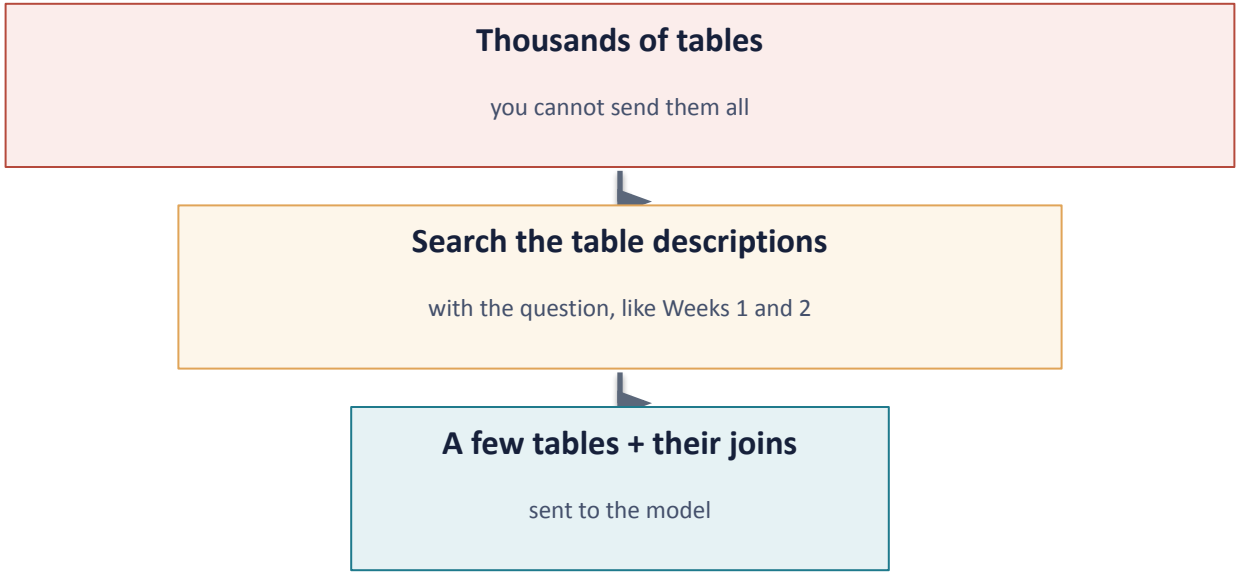
Worked examples

real questions with reviewed SQL

```
Q: Orders in March?
SQL: SELECT COUNT(*) ...
```

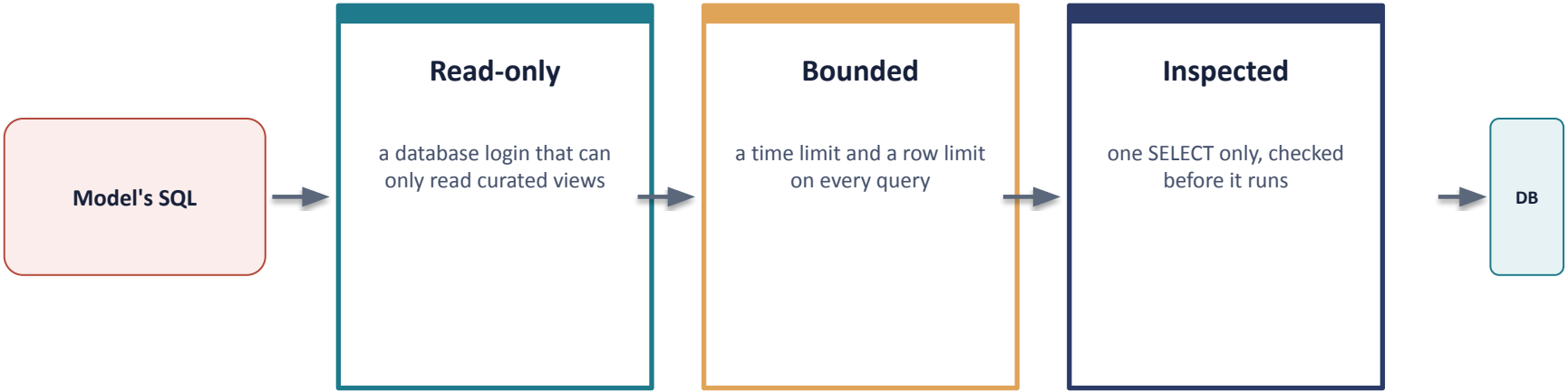
The schema comes free. Rules and examples are work, and they are where accuracy is won.

A real warehouse has thousands of tables. Send only a few



Finding the right tables is retrieval, the skill from Weeks 1 and 2.

Generated SQL is untrusted input: three gates

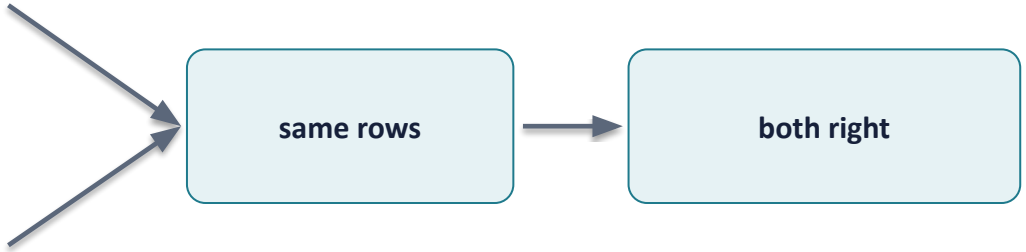


Yesterday's lesson again: the protection lives in code, not in the prompt.

Right means the same rows, not the same SQL

```
SELECT COUNT(*) FROM orders o
JOIN customers c ON ...
WHERE c.is_test = 0
```

```
SELECT COUNT(*) FROM orders
WHERE customer_id NOT IN
  (SELECT id FROM test_accounts)
```



Exact match compares the SQL text and would mark one of these wrong. **Execution accuracy** runs both and compares the answers.

Score on a set of real questions with reviewed answers. Lab 6 uses 16.

Sometimes the right answer is a question back

Ambiguous

"Show me top customers"

↓

"By revenue, by orders, or most recent?"

Not in the data

"Why did customers leave?"

↓

"The data has no reason for leaving."

Unclear dates

"Orders last month"

↓

"Which month do you mean?"

A refusal is a success. A confident wrong number is the failure.

QUICK CHECK

Three questions before the break

1

Your NL2SQL bot writes valid SQL every time and is still often wrong. Where do you look first?

2

Two queries look different but return the same rows. Is one of them wrong?

3

Name the three gates between generated SQL and your database.

Answers in chat. Wait for two people to commit before the next slide.

ANSWERS

Three questions before the break

1

What you send it

Missing business rules and examples, not the model. The meaning is the hard part.

2

No

Execution accuracy compares results. Same rows means both are right.

3

Read-only, bounded, inspected

A read-only login, time and row limits, and one SELECT only, checked in code.

Break

10 minutes. After the break: writing each definition once, for every tool.

3

Semantic layer

Write each business number once, and let every tool ask that place

Three teams, one word, three numbers

Finance

"Revenue last quarter"

```
revenue after refunds
```

Sales

"Revenue last quarter"

```
bookings when signed
```

Product

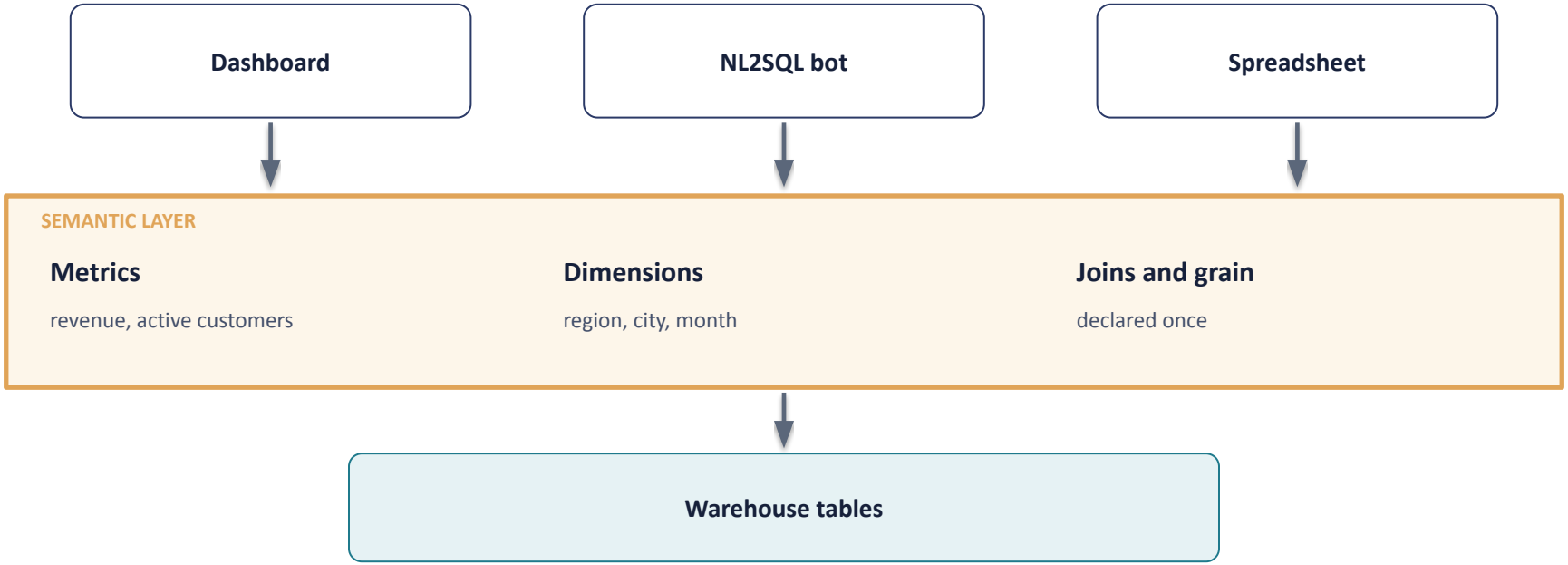
"Revenue last quarter"

```
revenue minus promotions
```

Nobody is lying. Each team's query has its own rule, in its own file. Every meeting starts by reconciling them.

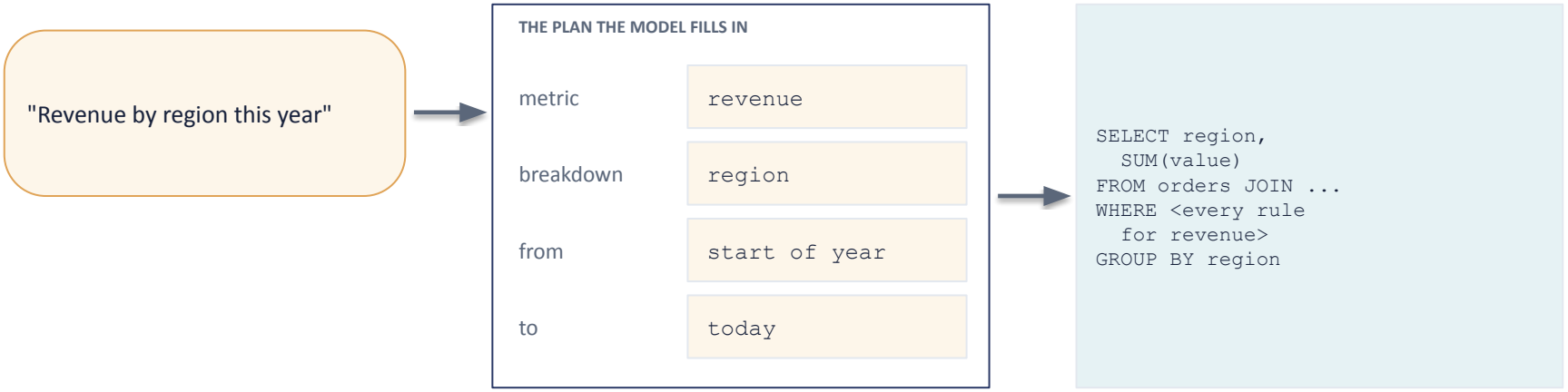
Rules in one prompt fix one bot. Rules in one shared place fix the company.

A semantic layer sits between the data and every tool



Every tool asks the layer, so every tool gets the same number.

The model fills in a form. Code writes the SQL



The plan has a "none" choice: if no metric fits, the layer refuses instead of guessing.

A much smaller decision for the model, and the rules can never be forgotten.

4

Narration and charts

The last step, where a right number is easiest to tell wrongly

Say what the data shows, never why

NARRATION

"Sales rose in March compared with February, for delivered orders only."

`the number` `the comparison` `the scope`

INVENTION

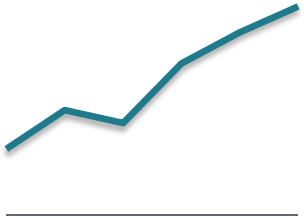
"Sales rose in March because of the new advert."

Nothing in the result says why. The cause is made up, and it sounds just as sure.

Build the sentence in code from the number. Leave no place for a cause.

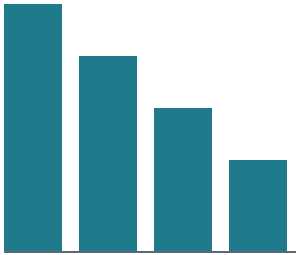
The shape of the question picks the chart

Over time



a line

Compare groups



sorted bars

Parts of a whole



a stacked bar, rarely a pie

One number

one figure, in words

no chart at all

Decide the chart in code from the question shape, not from what looks impressive.

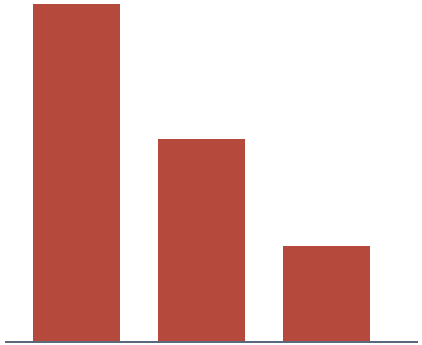
A correct chart can still mislead

Same data



bars start at zero

Same data



axis starts high

Missing denominator

"Complaints went up" may only mean orders went up. Show the rate, with its base.

Bars start at zero. Rates show their base. Both are rules for code.

5

Lab 6 on FreshCart

The same ideas on real data: one question, two numbers, and a fix

Lab 6 starts with one plain question



Do they agree? If not, why not, and what fixes it? That is the lab.

Lab 6

Questions in English, numbers you can defend

About 50 minutes · lab06_essentials notebook · do Lab 5 first

2	The model writes the SQL
3	Where do the extra orders come from?
4	Add business rules
5	Add worked examples
6	Score 16 questions, three ways
7	One phrase, two numbers
8	The semantic layer
9	A sentence and a chart

What Lab 6 found

Trainer runs, 14 October 2026. Compare with your own run.

LAB 6, STEPS 2 TO 5

Same question, three prompts, two confident numbers

A schema only

491

one run used the old orders table: 306

B + 8 rule lines

491

same query: the rule was ignored

C + 4 worked examples

450

matches the reviewed answer

The 41 extra orders: test accounts 1 and 2. 491 minus 41 is 450. The schema has is_internal; nothing says what it means.

A rule in the prompt is a request. An example showed the model what to do.

On 16 questions: rules helped, examples took it to 14



Wrong every time: N10 average delivery time, N16 orders "last month"

You only know which 14 because a person wrote 16 reviewed answers.

Refund rate: one phrase, two honest numbers

CLAIMS PER ORDER

0.0452

refund claims divided by orders

ORDERS WITH A CLAIM

0.0409

orders with at least one claim, divided by orders

March 2024, test accounts left out. They differ because some orders have more than one claim.

No prompt can settle this. A person decides once; the layer holds it.

The semantic layer: fewer answers, rules always right

6	4	10
answered	right	refused

BOTH MISTAKES WERE IN READING THE QUESTION

N07 "approved" claims the plan has no status, so the word was dropped

N16 "last month" guessed a month instead of refusing

SQL writer

answered 16, right 14

Semantic layer

answered 6, right 4, every rule applied

Layer without refuse rules

answered 9, right 4

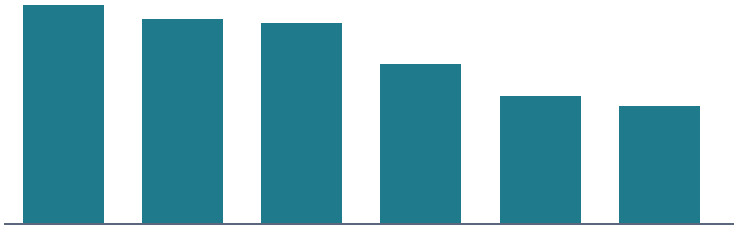
The trade: fewer answers, and every answer carries the agreed rules.

The last step, built in code

THE SENTENCE, BUILT FROM THE NUMBERS

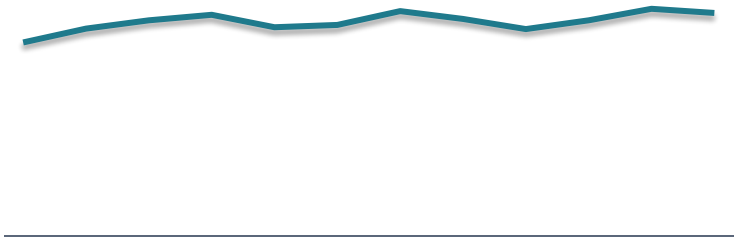
"Revenue was 531,043.03 in March 2024, up 1.7% from 522,100.56 in February 2024. Scope: delivered orders, excluding test accounts."

Revenue by city, Q2 2024 bars: places



as Lab 6 draws it, from the layer's numbers

Orders by month, 2024 a line: time



as Lab 6 draws it, from the layer's numbers

No cause, no extra currency sign, and the chart chosen by rule.

The week in one picture

Day 5: tools

The model asks, your code acts. Checks live in code.

Day 6: numbers

Rules and examples ground the SQL. A semantic layer holds the definitions.

Next: agents

Week 4: a model that plans several steps, built in Azure AI Foundry.

Priya can now cite the policy and defend the number. Next, an agent that does both.